VerdeBase (PlantCare) — Complete Project Explainer

Oct 20, 2026 · @CutuGoat

1. The project in one minute

VerdeBase (also called PlantCare 2.0) is a website that helps people look after their house plants, built so that all its data and most of its logic live inside a PostgreSQL database. That second part is what makes it a DBMS project.

Think of it as a diary for your plants. You add a plant (say, "Money Plant in the balcony"), and every time you water it, feed it fertilizer, measure it or notice a disease, you write it down in the app. The app then tells you how healthy each plant is, when to water next, sends email reminders, and shows charts of how your plants are doing.

The project has **two halves**:

Half	Who it is for	What it does
PlantCare app	Normal users (plant owners)	Add plants, log watering / fertilizer / growth / health / disease, see a dashboard, analytics, reminders, AI plant chatbot
DBMS Studio	Admin, examiner, teacher	Pages that open up the database and show its inside: tables, ER diagram, normalization, constraints, views, functions, triggers, transactions, indexes, query plans, audit log, a SQL Lab to run queries

Why both halves matter. The first half gives the database a real reason to exist (real users, real data). The second half lets you *prove* the DBMS concepts in front of the examiner while the system is running, instead of only talking about them.

In one line for the viva: "VerdeBase is a plant-care management system on a normalized 14-table PostgreSQL database, where business rules run inside the database as constraints, views, functions and triggers, and a built-in DBMS Studio lets us inspect all of it live."

Numbers to remember: 14 tables, 6 views, 5 stored functions, 5 business triggers (+5 small `updated_at` triggers), about 30 indexes, 2 user roles (admin, user).

2. Basic words you need first (simple language)

If you know these 20 words, every other part of this explainer will make sense.

Word	Simple meaning	Example from this project
Database	An organised store of data that a program can search and update safely	The database named `verdebase`
DBMS	The software that manages the database (stores, checks, searches, protects it)	PostgreSQL
PostgreSQL (Postgres)	A free, powerful relational DBMS	Runs on your laptop at `localhost:5432`
Table (relation)	Like an Excel sheet: fixed columns, many rows	`plants`, `watering_logs`
Row (tuple / record)	One entry in a table	One plant: "Tulsi, balcony, healthy"
Column (attribute)	One property every row has	`plant_name`, `health_status`
SQL	The language used to talk to the database	`SELECT * FROM plants WHERE user_id = 2;`
Primary key (PK)	A column whose value is unique for every row; the row's ID	`plant_id`
Foreign key (FK)	A column that points to the PK of another table	`plants.user_id` points to `users.user_id`
Constraint	A rule the database enforces; bad data is rejected	`health_score BETWEEN 0 AND 100`
View	A saved SELECT query you can read like a table	`plant_health_summary`
Stored function	A small program saved inside the database	`calculate_plant_health_score()`

Word	Simple meaning	Example from this project
Trigger	Code the database runs automatically when something happens	After a watering is inserted, update plant health
Index	A lookup structure (like a book index) that makes searches fast	`idx_plants_user_id`
Transaction	A group of steps that either all succeed or all get undone	`BEGIN ... COMMIT` / `ROLLBACK`
Frontend	The part you see in the browser (pages, buttons, charts)	`app/dashboard/page.jsx`
Backend	Code on the server that receives requests and talks to the database	`app/api/plants/route.js`
API	A fixed set of URLs the frontend calls to get or send data	`GET /api/plants`, `POST /api/watering`
JSON	The text format data travels in between frontend and backend	`{"plant_name": "Tulsi"}`
Server	The program that is running and answering requests	`npm run dev` starts it at `localhost:3000`

How they fit together, in one sentence: you click a button on the *frontend*, it calls an *API* URL on the *backend*, the backend sends *SQL* to *PostgreSQL*, PostgreSQL checks *constraints*, may fire *triggers*, stores *rows* in *tables*, and sends the answer back as *JSON* to show on screen.

3. What a user sees and does (page by page)

A user lands on the home page, signs up or logs in, and then moves through the left sidebar. The sidebar has 9 app pages for everyone, plus 16 DBMS Studio pages that only the admin sees.

The journey of a normal user

1. **Home page** (`/` , file `app/page.js`): a landing page with a 3D animated hero (Spline), the project name, and buttons to get started.
2. **Login / Register** (`/auth`): enter name, email and a password of at least 6 characters. A new account is always a normal user (role 2).
3. **Overview / Dashboard** (`/dashboard`): five stat cards (Total Plants, Healthy, Needs Attention, Critical, Tasks Today), the last 5 activities, a pie chart of plants by species, and a 30-day line of average health score.
4. **My Plants** (`/plants`): a grid of your plants with search and filters (by health status and growth stage). Here you add, edit and delete plants.
5. **Plant detail** (`/plants/[id]` , e.g. `/plants/7`): one plant's full profile, its statistics (total waterings, average health, height gained, next watering date) and five tabs: Watering, Fertilization, Growth, Health, Diseases. Each tab lists that plant's history and lets you add a new entry.
6. **Species** (`/species`): the plant encyclopedia. Each species shows its water and sunlight needs, soil, temperature range, difficulty and default care intervals. You can add a new species.
7. **Care** (`/care`): quick logging of watering and fertilizing for any plant, with the recent log list.
8. **Growth** (`/growth`): log height (cm), spread, leaf count and growth stage, and see growth over time.
9. **Health** (`/health`): log a health check with a score from 0 to 100, a status and symptoms.
10. **Reminders** (`/reminders`): set a care routine, e.g. "Water the Tulsi every Monday and Thursday at 08:00", with the email address to send it to.
11. **Analytics** (`/analytics`): pick a period (e.g. 7, 30, 90 days) and see totals plus charts of waterings per day, average health per day, and plants per species.
12. **AI Assistant** (`/chatbot`): ask plant-care questions in plain English and get short bullet-point answers.

What the admin sees in addition

The admin (role 1, seed login `admin@verdebase.com`) gets a second sidebar group, **DBMS Studio**: Demo Mode (Viva), Database Overview, ER Explorer, Tables, Relational Model, Normalization, SQL Lab, Constraints, Views, Functions, Triggers, Transactions, Indexes, Query Performance, Statistics, Audit Logs. Section 14 explains each one.

The admin also sees **every user's plants and logs**, while a normal user only ever sees their own. That rule is enforced in the SQL itself (section 12).

What happens behind a simple click

None of these pages store data themselves. Each page only *asks the backend* for data and *shows* it. For example, the Dashboard page calls `GET /api/dashboard`, and that one call runs four SQL queries: it reads the `user_dashboard_stats` view, the `plant_activity_summary` view, a GROUP BY on species, and a 30-day average of health scores.

4. Tech stack: every tool and why it is there

The whole project is one JavaScript codebase (Next.js) talking to one PostgreSQL database. Everything else is a helper library. The list comes from `package.json`.

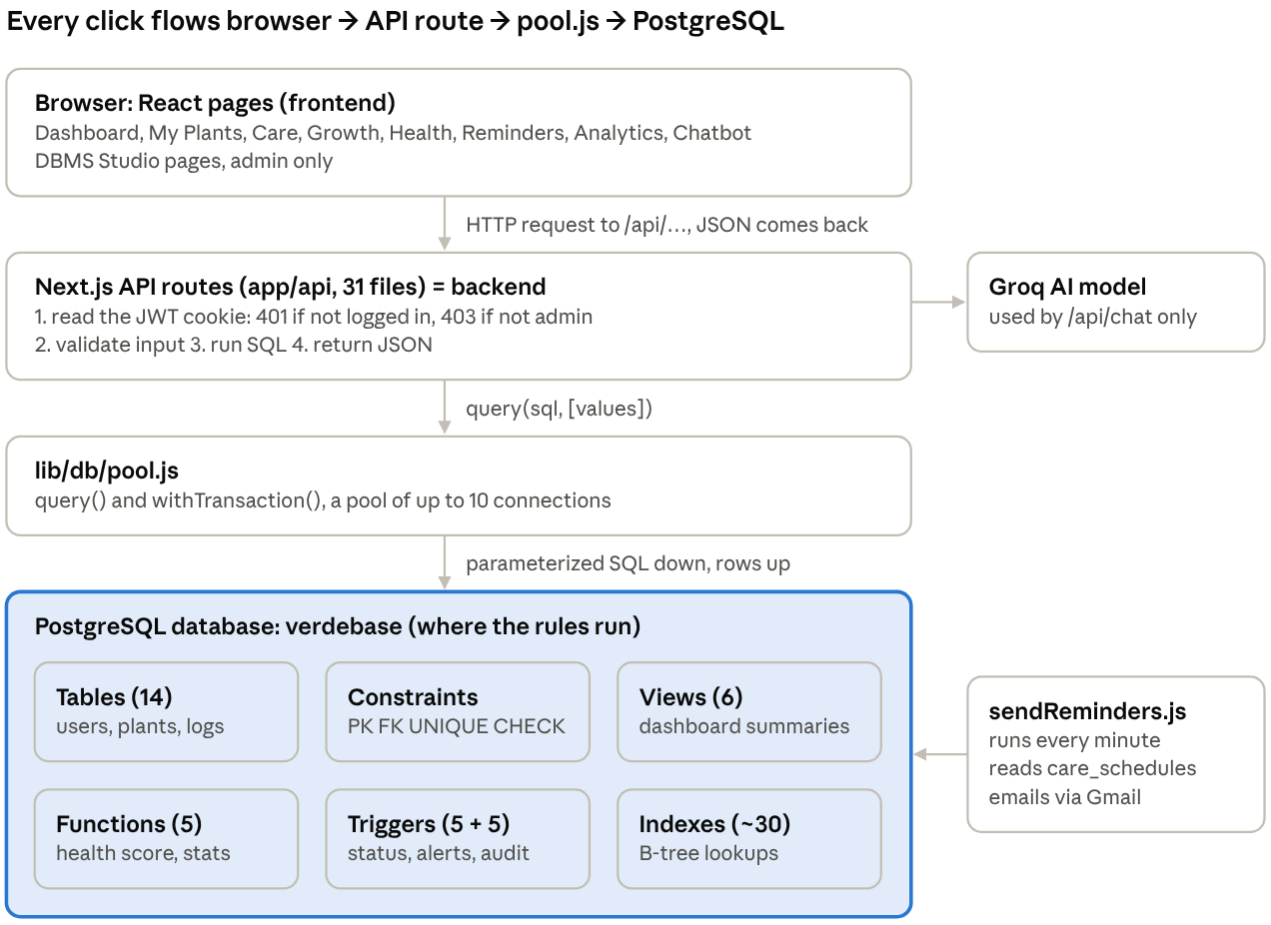
Layer	Tool	What it does here	Why it was chosen
Database	**PostgreSQL** (16/17)	Stores all data; runs constraints, views, functions, triggers	Free, strict, supports PL/pgSQL, triggers, JSONB, arrays, LATERAL joins, EXPLAIN ANALYZE
DB driver	**pg** (node-postgres)	Lets the JavaScript code send SQL to Postgres; gives a *connection pool*	Standard, supports `$1, $2` parameters (stops SQL injection)
Framework	**Next.js 15**	Serves both the web pages and the API in one project	No separate Express server needed; file name = URL
UI library	**React 19**	Builds the pages out of components (cards, modals, sidebar)	Next.js is built on React
Styling	**Tailwind CSS 4** + `globals.css`	Dark green "botanical" theme	Fast styling
Charts	**Recharts**	Pie, line, area and bar charts on Dashboard and Analytics	Simple React charts
ER diagram	**@xyflow/react** (React Flow)	Draws the draggable ER diagram in DBMS Studio	Boxes and arrows out of the box
Icons	**lucide-react**, **react-icons**	All icons	

Layer	Tool	What it does here	Why it was chosen
Animation	**framer-motion**, **gsap**, **Spline**	Page animations and the 3D hero on the home page	Looks
Passwords	**bcryptjs**	Turns passwords into one-way hashes before saving	Never store plain passwords
Login sessions	**jose** (JWT)	Creates a signed token stored in a cookie	Stateless login, no session table needed
Email	**nodemailer**	Sends reminder emails through Gmail	
Scheduler	**node-cron**	Runs the reminder check every minute	
AI chat	**groq-sdk**	Sends chat messages to an AI model (`openai/gpt-oss-20b` hosted on Groq)	Fast, free tier
Config	**dotenv**	Reads passwords and keys from the `.env` file	Keeps secrets out of code
Legacy	`@supabase/supabase-js`	Left over from version 1, which used Supabase (hosted Postgres). Version 2 replaced it with a direct Postgres connection	Can be removed

One-line answer if asked "what is your stack?": Next.js and React on the front, Next.js API routes on the back, PostgreSQL as the database through node-postgres, with JWT + bcrypt for login.

5. How the pieces connect

The project is a three-tier system: the browser shows pages, the Next.js API routes are the backend, and PostgreSQL holds the data and most of the rules. The browser never talks to the database directly; everything goes through an API route and the single connection file `lib/db/pool.js`.



VerdeBase architecture · 4 layers + 2 side services

Two side services hang off the main path: the chatbot route calls Groq's AI model, and a separate reminder worker reads `care_schedules` every minute and sends emails.

The journey of one click, in 8 steps

1. **You click** a button, e.g. "Log watering".
2. **The page (React)** calls `fetch('/api/watering', {method: 'POST', body: JSON})`. The login cookie travels with it automatically.
3. **Next.js** sees the URL `/api/watering` and runs the `POST` function in `app/api/watering/route.js`.
4. **The route checks who you are** by verifying the JWT in the cookie (401 if not logged in) and checks the input (400 if bad).
5. **The route calls `query(sql, values)`** from `lib/db/pool.js`. The pool lends it one of up to 10 open connections to PostgreSQL.
6. **PostgreSQL does its work:** checks constraints, writes the row, fires triggers, which call functions and update other tables.
7. **Rows come back** to the route, which returns them as JSON with a status code (201 = created).

8. **The page updates** what you see. Lists and the dashboard read from views, so they are already up to date.

Three-tier names for the viva: presentation tier = React pages; application tier = Next.js API routes; data tier = PostgreSQL.

6. Folder and file map

Three folders matter: `database/` (all SQL), `lib/` (shared helpers), and `app/` (pages + API). In Next.js the folder path *is* the URL: `app/plants/page.jsx` is the page at `/plants`, and `app/api/plants/route.js` is the API at `/api/plants`.

```
plantcare/
├── database/              ← THE DBMS PART (pure SQL)
|   ├── schema.sql         14 tables, constraints, updated_at triggers,
default roles
|   ├── indexes.sql        ~30 indexes
|   ├── views.sql          6 views
|   ├── functions.sql      5 stored functions (PL/pgSQL)
|   ├── triggers.sql       5 business triggers
|   ├── seed.sql           demo data (users, species, plants, logs)
|   ├── sample_queries.sql practice queries (joins, aggregates, subqueries)
|   ├── demo_queries.sql   queries to run live in the viva
|   └── init.js            runs all the files above in the right order
├── lib/
|   ├── db/pool.js         the ONE place that connects to Postgres: query(),
withTransaction()
|   ├── auth/session.js    create / verify JWT, set / read / clear the cookie
|   ├── auth/middleware.js withAuth / withAdmin wrappers
|   ├── useAuth.js         React hook: "who is logged in?" for pages
|   └── supabaseClient.js  leftover from v1, unused
├── app/
|   ├── page.js            home / landing page
|   ├── layout.js          wraps every page (fonts, global CSS)
|   ├── globals.css        colours and theme
|   ├── auth/ dashboard/ plants/ plants/[id]/ species/ care/ growth/
|   |   health/ reminders/ analytics/ chatbot/          ← user pages
|   ├── dbms/…             16 DBMS Studio pages (admin)
|   ├── components/        Sidebar, AppShell, StatCard, Modal, HealthBadge,
LoadingSpinner, hero, navbar
|   └── api/               ← BACKEND: 31 route files (section 13)
├── sendReminders.js       background worker: every minute, emails due
```

```
reminders
├── .env / .env.example    DB password, JWT secret, Gmail login, Groq key
├── package.json           list of libraries + scripts (dev, build, db:setup)
└── README.md              project documentation
```

Files you should be able to open and explain in the viva, in order of importance: `schema.sql`, `triggers.sql`, `functions.sql`, `views.sql`, `indexes.sql`, `lib/db/pool.js`, `app/api/watering/route.js`, `app/api/auth/login/route.js`.

The folders `node_modules/` (downloaded libraries) and `.next/` (build output) are generated automatically. You never edit them and they are not your code.

7. The database: all 14 tables explained

The database `verdebase` has 14 tables in four groups: lookup tables, core entities, activity logs, and system tables. Every table has an auto-numbered integer primary key (`SERIAL`) and a `created_at` timestamp. All of it is in `database/schema.sql`.

A few data types you will see:

- `SERIAL`: an integer that fills itself in as 1, 2, 3...
- `VARCHAR(n)`: text up to n characters. `TEXT`: text of any length.
- `INTEGER`: whole number. `NUMERIC(8,2)`: decimal number like 123456.78.
- `BOOLEAN`: true or false. `DATE`: a day. `TIME`: a clock time. `TIMESTAMPTZ`: date + time + timezone.
- `TEXT[]`: an **array** (list) of text inside one cell, e.g. `{monday,thursday}`.
- `JSONB`: a JSON object stored in binary form, e.g. `{"plant_id": 3, "severity": "severe"}`.
- `INET`: an IP address.

Group A: lookup (reference) tables

Data that is written once and reused by many rows.

1. `roles`: the kinds of user.

Column	Type	Meaning
role_id	SERIAL PK	1 = admin, 2 = user
role_name	VARCHAR(50), UNIQUE, NOT NULL	'admin' or 'user'
description	TEXT	What the role may do

2. `species` : the plant encyclopedia (12 seeded, e.g. Monstera, Fiddle Leaf Fig, Snake Plant, Pothos, Aloe vera).

Column	Type	Meaning
species_id	SERIAL PK	ID
common_name	VARCHAR, NOT NULL	"Monstera"
scientific_name	VARCHAR, NOT NULL, UNIQUE	"Monstera deliciosa" (a natural key)
family	VARCHAR	"Araceae"
water_requirement	CHECK in (low, moderate, high)	How thirsty
sunlight_requirement	CHECK in (full_sun, partial_shade, indirect, low_light)	How much light
soil_type	VARCHAR	Recommended soil
min_temp_celsius, max_temp_celsius	NUMERIC(5,2)	Safe temperature range; CHECK min ≤ max
care_difficulty	CHECK in (beginner, intermediate, expert)	
description	TEXT	

3. `care_profiles` : default care routine of a species. Split out of `species` for 3NF (section 9).

Column	Type	Meaning
profile_id	SERIAL PK	
species_id	FK → species, **UNIQUE**, ON DELETE CASCADE	UNIQUE makes this **one-to-one** with species
watering_interval_days	INTEGER, CHECK 1–90, default 7	Water every N days
fertilize_interval_days	INTEGER, CHECK 1–365, default 30	Feed every N days
fertilizer_type	VARCHAR	e.g. "Balanced liquid 20-20-20"

Column	Type	Meaning
repot_interval_months	INTEGER, CHECK 1–60	
pruning_notes	TEXT	

4. `diseases` : catalog of plant diseases (e.g. Leaf Spot, Root Rot, Spider Mites).

Column	Type	Meaning
disease_id	SERIAL PK	
disease_name	VARCHAR, UNIQUE	
category	CHECK in (fungal, bacterial, viral, pest, nutritional, environmental)	
description, symptoms, treatment, prevention	TEXT	Encyclopedia text
severity	CHECK in (mild, moderate, severe, critical)	Typical severity

Group B: core entities

5. `users` : people who log in.

Column	Type	Meaning
user_id	SERIAL PK	
email	VARCHAR(255), NOT NULL, UNIQUE, CHECK regex	Login name; the regex rejects "abc" or "a@b"
password_hash	VARCHAR(255)	bcrypt hash, never the real password
full_name	VARCHAR(150)	
role_id	FK → roles, default 2, ON DELETE RESTRICT	
avatar_url	VARCHAR	Profile picture
is_active	BOOLEAN	Account enabled?
last_login_at	TIMESTAMPTZ	Set on every login

Column	Type	Meaning
updated_at	TIMESTAMPTZ	Set by a trigger on every update

6. **`plants`** : the centre of the whole schema. Every log table points here.

Column	Type	Meaning
plant_id	SERIAL PK	
user_id	FK → users, NOT NULL, ON DELETE CASCADE	Owner
species_id	FK → species, nullable, ON DELETE SET NULL	What kind of plant
plant_name	VARCHAR(200), NOT NULL	"My Monstera"
nickname	VARCHAR(100)	"Monty"; UNIQUE together with user_id
location	VARCHAR	"Living Room"
date_acquired	DATE	
growth_stage	CHECK in (seedling, juvenile, mature, flowering, dormant)	Updated by trigger
health_status	CHECK in (healthy, needs_attention, critical, recovering)	Updated by triggers
image_url, notes		
is_active	BOOLEAN	FALSE = deleted (soft delete)

Group C: activity logs (the time series)

Each is **one-to-many** from `plants` : one plant, many log rows. Each has `plant_id` FK → plants ON DELETE CASCADE.

Table	Important columns and their CHECKs
7. **`watering_logs`**	watered_at, quantity_ml (> 0), method (hand_watering, drip, spray, bottom_watering, self_watering), notes

Table	Important columns and their CHECKs
8. `fertilization_logs`	fertilizer_type (NOT NULL), quantity_grams (> 0), applied_at, method (soil, foliar, liquid, slow_release)
9. `growth_logs`	height_cm (> 0), spread_cm (> 0), leaf_count (≥ 0), growth_stage, recorded_at
10. `health_logs`	health_status, health_score (0–100, NOT NULL), symptoms (**TEXT[] array**), recorded_at

11. `disease_records` : the **junction table** that makes plants ↔ diseases many-to-many.

Column	Type	Meaning
record_id	SERIAL PK	
plant_id	FK → plants, ON DELETE CASCADE	Which plant
disease_id	FK → diseases, ON DELETE RESTRICT	Which disease
detected_at, resolved_at	TIMESTAMPTZ	CHECK resolved_at ≥ detected_at; resolved_at NULL = still sick
severity	CHECK (mild … critical)	Severity *in this case*
treatment_used, notes	TEXT	

Group D: system tables

12. `care_schedules` : reminders. Columns: user_id (FK), plant_id (FK, optional), task_type (watering, fertilizing, repotting, pruning, inspection, custom), schedule_time (TIME), schedule_days (TEXT[] like `{monday,thursday}`), is_active, last_sent_at, email.

13. `notifications` : in-app alerts. Columns: user_id, plant_id (ON DELETE SET NULL), title, message, type (watering_due, health_alert, disease_detected, care_reminder, system, growth_milestone), is_read. **Nobody inserts here by hand: the triggers write these rows.**

14. `audit_logs` : the history book of the system. Columns: user_id (ON DELETE SET NULL, so history survives a deleted user), action (INSERT, UPDATE, DELETE, LOGIN, LOGOUT, REGISTER), entity (table name), entity_id, old_data / new_data (**JSONB**

snapshots), ip_address (INET), user_agent. Rows come from triggers and from the login route.

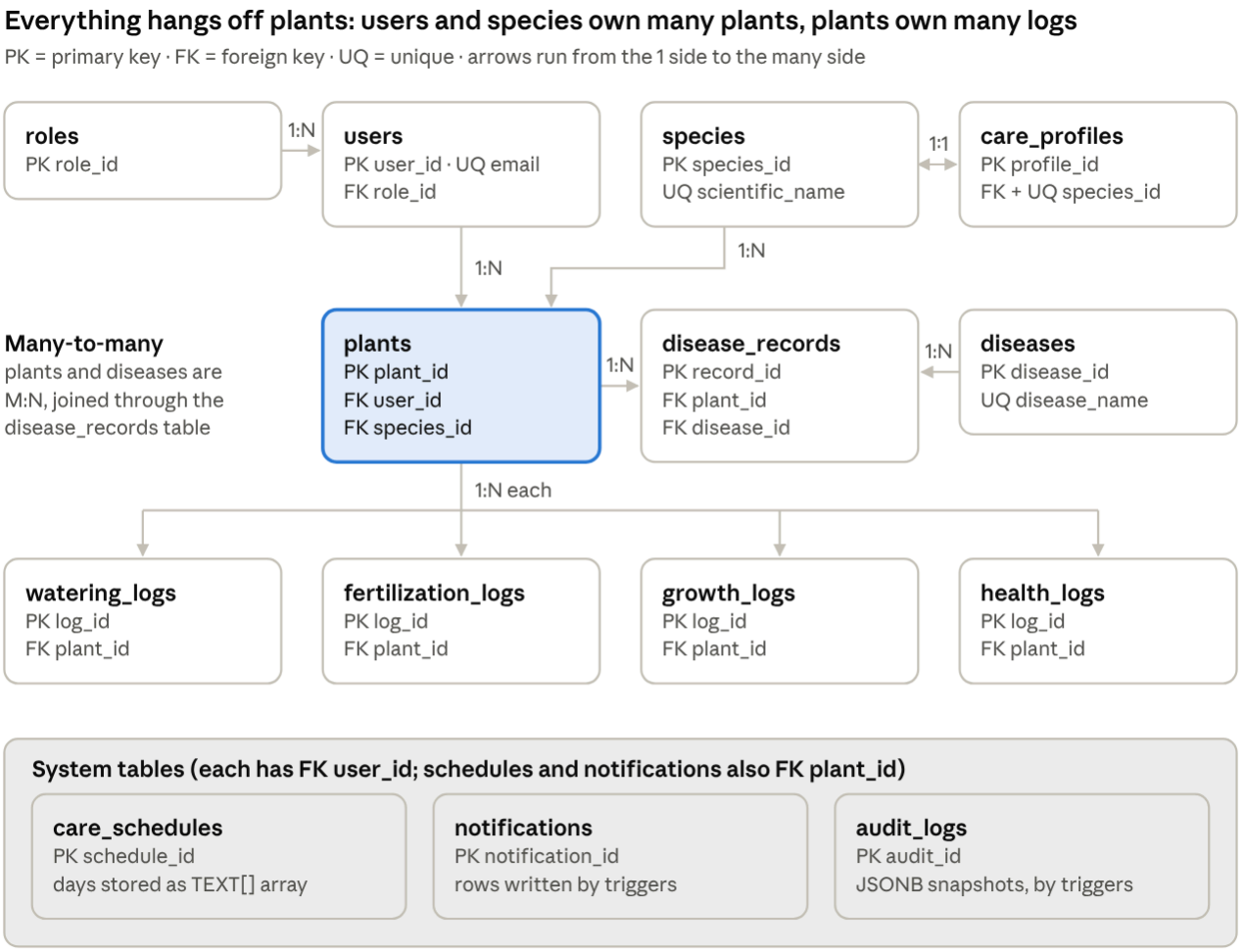
Seed data (`seed.sql`)

The seed script is wrapped in `BEGIN; … COMMIT;` so it loads all-or-nothing. It creates 2 users (`admin@verdebase.com` / `Admin@123`, and `jane@verdebase.com` / `User@123`), 12 species with care profiles, 8 plants for Jane (Monty, Fig the Diva, Snakey, Goldie, Lily, Al, ZeZe, Spidey), watering/fertilizing/growth/health logs over the last 30 days, a disease catalog, disease records, and care schedules. It does **not** insert notifications or audit logs: the triggers create those while the seed runs, which is itself a demo of triggers.

The seed links rows by **natural keys** (email, scientific_name, nickname) with JOINs instead of hard-coding IDs like 1, 2, 3.

8. Relationships, keys and constraints

Everything hangs off `plants`: a user owns many plants, a species describes many plants, and each plant has many log rows. The database itself enforces these links, so bad data cannot get in even if the website has a bug.



ER diagram · 14 tables, keys and cardinalities

`plants` sits in the middle: two parents above it (users, species), five kinds of child rows below it, and diseases joined to it many-to-many through `disease_records`.

Relationships (cardinality)

Relationship	Type	How it is built
roles → users	1 : N	`users.role_id` FK
users → plants	1 : N	`plants.user_id` FK
species → plants	1 : N (optional)	`plants.species_id` FK, nullable
species ↔ care_profiles	**1 : 1**	`care_profiles.species_id` is FK **and UNIQUE**
plants → watering / fertilization / growth / health logs	1 : N each	`plant_id` FK in each log table

Relationship	Type	How it is built
plants ↔ diseases	**M : N**	Through the junction table `disease_records`
users → care_schedules, notifications, audit_logs	1 : N	`user_id` FK
plants → care_schedules, notifications	1 : N (optional)	`plant_id` FK

Strong vs weak entities (viva favourite). The log tables behave like weak entities: a watering log means nothing without its plant, and it is deleted along with the plant (CASCADE). Technically they have their own surrogate key (`log_id`), so in the schema they are stored as regular tables that depend on `plants` through a foreign key.

Kinds of keys used

- **Primary key:** every table, all `SERIAL` integers (surrogate keys).
- **Foreign key:** 15 FK links in total, listed above.
- **Candidate / natural keys** (enforced with UNIQUE): `users.email` , `species.scientific_name` , `diseases.disease_name` , `roles.role_name` .
- **Composite unique key:** `UNIQUE (user_id, nickname)` on plants. Two different users can both name a plant "Monty", but one user cannot have two "Monty"s.

Constraints: the five types and where each is used

Constraint	What it guarantees	Examples in this project
PRIMARY KEY	Each row has a unique, non-null ID	`plant_id` , `user_id` , …
NOT NULL	Value must be given	`plants.plant_name` , `health_logs.health_score`
UNIQUE	No duplicates	`users.email` , `species.scientific_name` , `(user_id, nickname)`
CHECK	Value must satisfy a condition	`health_score BETWEEN 0 AND 100` ; `quantity_ml > 0` ; `watering_interval_days BETWEEN 1 AND 90` ; `min_temp ⩽ max_temp` ; `resolved_at ⩾ detected_at` ; email regex; all the "IN (…)" value lists

Constraint	What it guarantees	Examples in this project
FOREIGN KEY	Referenced row must exist	`plants.user_id → users`
DEFAULT	Fills a value if none is given	`health_status = 'healthy'`, `created_at = NOW()`, `role_id = 2`

What happens on delete (referential actions)

FK	Action	Meaning in plain words
plants.user_id → users	ON DELETE CASCADE	Delete a user: their plants are deleted too
all log tables.plant_id → plants	ON DELETE CASCADE	Delete a plant: all its logs go too
care_profiles.species_id → species	ON DELETE CASCADE	Delete a species: its care profile goes too
plants.species_id → species	ON DELETE SET NULL	Delete a species: plants stay, their species becomes empty
notifications.plant_id, audit_logs.user_id	ON DELETE SET NULL	Keep the history, just lose the link
users.role_id → roles	ON DELETE RESTRICT	You cannot delete a role while users have it
disease_records.disease_id → diseases	ON DELETE RESTRICT	You cannot delete a disease that is recorded on a plant

What the error looks like

If anyone tries `INSERT INTO health_logs (plant_id, health_status, health_score) VALUES (1, 'healthy', 150);`, PostgreSQL refuses with error code **23514** (check_violation). A duplicate email gives **23505** (unique_violation), and the register route catches exactly that code to show "An account with this email already exists". A missing parent row gives **23503** (foreign_key_violation).

9. Normalization: from one messy sheet to 3NF

Normalization means splitting data into tables so that each fact is stored in exactly one place. That removes the three anomalies (insert, update, delete). Here is how this project's design is reached, step by step, from an imaginary single spreadsheet.

Step 0: the unnormalized table (UNF)

Imagine a beginner keeps everything in one sheet:

owner_email	owner_name	plant_name	species	scientific_name	water_every_days	waterings	diseases
jane@…	Jane	My Monstera	Monstera	Monstera deliciosa	7	01-Oct, 08-Oct, 15-Oct	Leaf Spot
jane@…	Jane	Office ZZ	ZZ Plant	Zamioculcas zamiifolia	14	03-Oct	Root Rot, Spider Mites
ravi@…	Ravi	Big Monstera	Monstera	Monstera deliciosa	7	05-Oct, 12-Oct	

Problems:

- **Update anomaly:** change Monstera's watering interval from 7 to 6 and you must edit every Monstera row. Miss one and the data disagrees with itself.
- **Insert anomaly:** you cannot add a new species (say, Aloe vera) until someone owns one.
- **Delete anomaly:** delete Ravi's only plant and you lose his name and email too.
- **Non-atomic cells:** "01-Oct, 08-Oct, 15-Oct" is a list squeezed into one cell.

Step 1: First Normal Form (1NF): one value per cell

Rule: every cell holds one atomic value; no repeating groups.

The list of waterings becomes one row per watering, and the list of diseases becomes one row per disease. That is why `watering_logs`, `fertilization_logs`, `growth_logs`, `health_logs` and `disease_records` exist as separate tables, with one row per event.

Step 2: Second Normal Form (2NF): no partial dependency

Rule: be in 1NF, and every non-key column must depend on the **whole** key, not part of it.

After 1NF, a watering row would be keyed by (plant, watered_at). But `species`, `owner_email` and `plant_name` depend only on the *plant*, not on the watering date. That is a partial dependency. So the plant's own facts move to the `plants` table, and the log keeps only `plant_id` + its own columns (`watered_at`, `quantity_ml`, `method`). The owner's facts move to `users`.

Step 3: Third Normal Form (3NF): no transitive dependency

Rule: be in 2NF, and no non-key column may depend on another non-key column.

The functional dependencies are:

```
\text{plant\_id} \rightarrow \text{user\_id},\ \text{plant\_name},\
\text{species\_id},\ \text{location}
```

```
\text{species\_id} \rightarrow \text{common\_name},\
\text{scientific\_name},\ \text{family}
```

```
\text{species\_id} \rightarrow \text{watering\_interval\_days},\
\text{fertilizer\_type}
```

So `plant_id → species_id → watering_interval_days` is a **transitive dependency**: the interval depends on the plant only *through* the species. The fix: species facts go to `species`, and care facts go to `care_profiles`, keyed by `species_id`. Same for `users → role_id → role_name`: roles get their own table, and diseases get their own catalog table.

The result

Fact	Stored once in
A person's email and name	`users`
What a role is called	`roles`
What a Monstera is	`species`
How often to water a Monstera	`care_profiles`
What Root Rot is and how to treat it	`diseases`
Jane owns "My Monstera" in the living room	`plants`
"My Monstera" was watered on 8 Oct with 300 ml	`watering_logs`
"Office ZZ" got Root Rot on 3 Oct, severe	`disease_records`

Now changing Monstera's interval is **one UPDATE on one row** in `care_profiles`, a species can exist with no plants, and deleting a plant never deletes its owner.

Why is `care_profiles` separate from `species`, if it is 1 : 1? Strictly, a 1 : 1 split is not required by 3NF (both depend only on species_id). The project splits it to separate *what a plant is* (botany) from *how to care for it* (care rules), keep the species table small, and let care rules be edited without touching taxonomy. Say this honestly if asked.

Deliberate exceptions (denormalization): `plants.health_status` and `plants.growth_stage` could be derived from the latest logs, but are stored on the plant for fast reads and kept up to date by triggers. `health_logs.symptoms` is a `TEXT[]` array, which is technically not atomic (strict 1NF), and was chosen for simplicity.

10. Views, stored functions and triggers (the "smart" database)

This is the part that makes it a real DBMS project: the database does work by itself instead of only storing rows. Views are saved questions, functions are saved calculations, and triggers are automatic reactions.

10.1 Views (`database/views.sql`): saved queries

A view stores a SELECT, not data. Every time you read it, Postgres runs the query on the current data, so it is always up to date. The website reads views as if they were tables: `SELECT * FROM user_dashboard_stats WHERE user_id = $1`.

View	What it answers	SQL features it shows	Used by
`plant_health_summary`	For each plant: latest health score, last watering time, latest height	`LEFT JOIN LATERAL (… ORDER BY … DESC LIMIT 1)`	Plant lists, DBMS Studio Views page
`plant_care_summary`	For each plant: total waterings, fertilizations, logs, average health, first and current height	4 LEFT JOINs + `COUNT(DISTINCT)`, `MAX`, `AVG`, `GROUP BY`	Care pages
`disease_summary`	Every disease case, "active" or "resolved", days taken to heal	JOIN 3 tables + `CASE WHEN` + date `EXTRACT`	Health module
`plant_activity_summary`	One combined timeline of all care events	`UNION ALL` of 4 SELECTs	Dashboard "Recent activity"
`species_care_overview`	Each species + its care profile + how many plants use it	LEFT JOIN + `COUNT` + GROUP BY	Species page

View	What it answers	SQL features it shows	Used by
`user_dashboard_stats`	Per user: total plants, healthy, needs attention, critical, waterings this week, unread notifications	`COUNT(…) FILTER (WHERE …)`	Dashboard stat cards

What is a LATERAL join, simply? A normal subquery in FROM cannot see the outer row. `LATERAL` lets it: for *each* plant `p`, it runs "give me the newest health log *of this plant*". It is the cleanest way to get "latest row per group".

```
LEFT JOIN LATERAL (
    SELECT health_score, recorded_at, symptoms
    FROM health_logs
    WHERE plant_id = p.plant_id      -- refers to the outer row
    ORDER BY recorded_at DESC
    LIMIT 1
) hl ON TRUE
```

Why `COUNT(DISTINCT …)` in `plant_care_summary`? Joining a plant to 4 log tables multiplies rows (5 waterings × 3 fertilizations = 15 rows). DISTINCT counts each log once despite the multiplication.

10.2 Stored functions (`database/functions.sql`): calculations inside the database

Written in **PL/pgSQL**, Postgres's own programming language (variables, IF/ELSE, loops). The website calls them like `SELECT * FROM get_plant_statistics(7)`.

1. `calculate_plant_health_score(plant_id) → INTEGER`: the brain of the project. Step by step:

1. Start at **100**.
2. Look up this plant's species watering interval from `care_profiles` (7 days if none).
3. Days since last watering: if more than **2×** the interval, **−30**; else if more than the interval, **−15**. Never watered counts as 999 days.
4. Days since last fertilizing: if more than 60, **−10**.
5. If there is a manual health check, blend: **score = 60% of the computed score + 40% of the latest manual score**.
6. Each disease not yet resolved: **−15**.
7. Clamp to 0–100 with `GREATEST(0, LEAST(100, score))`.

Worked example: a Monstera (interval 7) last watered 10 days ago (−15 → 85), fertilized 20 days ago (no change), last manual score 70 (0.6 × 85 + 0.4 × 70 = 79), one active disease (−15) → **64**, which maps to "needs_attention".

2. `calculate_next_watering_date(plant_id) → TIMESTAMPTZ`: last watering + species interval. Never watered? Returns NOW() ("water it now").

3. `get_plant_statistics(plant_id) → TABLE(…)`: returns one row with 10 numbers: total waterings, fertilizations, growth logs, health logs, active diseases, average health score, height gained (MAX − MIN height), days since watered, next watering date, computed health. It calls functions 1 and 2 inside it. Used by the plant detail page.

4. `get_user_care_summary(user_id, days = 30) → TABLE(…)`: activity totals for one user over the last N days (plants watered, events, new and resolved diseases, average health). Shows a **parameter with a default value**. Used by Analytics.

5. `get_database_statistics()`: reads Postgres's own catalog tables to report row counts and sizes. Used by DBMS Studio.

There is also a small helper, `update_updated_at_column()`, in `schema.sql`.

10.3 Triggers (`database/triggers.sql`): automatic reactions

A trigger = **event + timing + function**. "AFTER INSERT ON watering_logs FOR EACH ROW, run fn_after_watering_insert()". Inside the function, `NEW` is the row that was just inserted.

#	Trigger	When	What it does automatically
1	`trg_after_watering_insert`	AFTER INSERT on `watering_logs`	Calls `calculate_plant_health_score()`. Sets `plants.health_status`: ⩾ 75 healthy, ⩾ 45 needs_attention, else critical. Writes an `audit_logs` row with a JSONB snapshot (plant, time, computed health)
2	`trg_after_disease_record_insert`	AFTER INSERT on `disease_records`	Inserts a "⚠️ Disease Detected: Root Rot" notification. Severe/critical → plant becomes critical; moderate → needs_attention. Writes an audit row
3	`trg_after_growth_log_insert`	AFTER INSERT on `growth_logs`	If the logged stage differs from the plant's stage: update `plants.growth_stage` and add a "🌱 Growth Milestone!" notification. Always writes an audit row

#	Trigger	When	What it does automatically
4	`trg_after_health_log_insert`	AFTER INSERT on `health_logs`	Maps the score to a status and updates the plant. If score < 30, adds a "🚨 Critical Health Alert!" notification
5	`trg_after_plant_insert`	AFTER INSERT on `plants`	Adds a "🌿 New Plant Added!" welcome notification and an audit row
6–10	`trg_<table>_updated_at` on users, species, care_profiles, plants, care_schedules	**BEFORE** UPDATE	Sets `NEW.updated_at = NOW()` before the row is saved

The watering trigger, shortened:

```
CREATE OR REPLACE FUNCTION fn_after_watering_insert() RETURNS TRIGGER AS $$
DECLARE v_computed_health INTEGER; v_user_id INTEGER;
BEGIN
    SELECT user_id INTO v_user_id FROM plants WHERE plant_id = NEW.plant_id;
    v_computed_health := calculate_plant_health_score(NEW.plant_id);
    IF v_computed_health >= 75 THEN
        UPDATE plants SET health_status = 'healthy' WHERE plant_id =
NEW.plant_id;
    ELSIF v_computed_health >= 45 THEN
        UPDATE plants SET health_status = 'needs_attention' WHERE plant_id =
NEW.plant_id;
    ELSE
        UPDATE plants SET health_status = 'critical' WHERE plant_id =
NEW.plant_id;
    END IF;
    INSERT INTO audit_logs (user_id, action, entity, entity_id, new_data)
    VALUES (v_user_id, 'INSERT', 'watering_logs', NEW.log_id::TEXT,
            jsonb_build_object('plant_id', NEW.plant_id, 'computed_health',
v_computed_health));
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_after_watering_insert
    AFTER INSERT ON watering_logs
    FOR EACH ROW EXECUTE FUNCTION fn_after_watering_insert();
```

Triggers can chain. The watering trigger runs `UPDATE plants`, and that UPDATE fires the BEFORE UPDATE trigger on `plants`, which sets `updated_at`. One INSERT from the website causes 1 insert + 1 update + 1 audit insert, all inside the same transaction: if any step fails, all are undone.

AFTER vs BEFORE. BEFORE triggers can change the row before it is saved (`updated_at`). AFTER triggers run once the row exists, so they can safely use its new ID and touch other tables (notifications, audit, plants).

Row-level vs statement-level. All triggers here are `FOR EACH ROW` : they run once per inserted row, not once per INSERT statement.

11. Indexes, query plans and transactions

Indexes make reads fast, EXPLAIN ANALYZE proves it, and transactions make writes safe. These are the "physical" and "reliability" parts of the DBMS syllabus.

11.1 Indexes (`database/indexes.sql`)

Without an index, finding Jane's plants means reading every row of `plants` (a **sequential scan**). An index is a sorted structure, a **B-tree** by default in Postgres, that jumps straight to the matching rows, like the index at the back of a textbook.

Free indexes: Postgres builds an index automatically for every PRIMARY KEY and every UNIQUE constraint.

Indexes this project adds (about 30), in four kinds:

Kind	Example	Why
Single-column on a foreign key	`idx_plants_user_id ON plants(user_id)` , `idx_watering_logs_plant_id`	Postgres does **not** auto-index FKs; JOINs and "WHERE user_id = ?" need them
Composite (multi-column)	`idx_watering_logs_plant_date ON watering_logs(plant_id, watered_at DESC)`	Exactly matches "latest watering of this plant", which the LATERAL views and health function ask constantly
Composite for a filter	`idx_plants_user_active ON plants(user_id, is_active)`	Every page asks "this user's active plants"
Partial index	`idx_notifications_user_unread ON notifications(user_id, is_read) WHERE is_read = FALSE`	Only indexes unread rows, so it is small; used for the unread badge
Sorting	`idx_audit_logs_created_at ON audit_logs(created_at DESC)`	"Newest first" lists

Composite index rule (leftmost prefix): an index on `(plant_id, watered_at)` helps queries that filter on `plant_id`, or on `plant_id` and `watered_at`, but **not** on `watered_at` alone. That is why there is a separate `idx_watering_logs_watered_at` too.

The cost of indexes: every INSERT and UPDATE must also update each index, and they take disk space. So you index columns that are searched often, not every column.

11.2 EXPLAIN and EXPLAIN ANALYZE

- `EXPLAIN SELECT …` shows the **plan**: how Postgres *intends* to run the query (Seq Scan, Index Scan, Hash Join, Nested Loop, Sort) and its estimated cost.
- `EXPLAIN ANALYZE SELECT …` actually runs it and shows the **real** time and row counts at each step.

The Query Performance page sends your SELECT to `/api/db/explain`, which runs `EXPLAIN (ANALYZE, FORMAT JSON)` and draws the plan as a tree. Because ANALYZE really executes the query, the route wraps it in `BEGIN … ROLLBACK` on one connection, so nothing can be changed.

Try this in the viva:

```
EXPLAIN ANALYZE
SELECT * FROM watering_logs
WHERE plant_id = 3
ORDER BY watered_at DESC
LIMIT 1;
```

You should see an **Index Scan using idx_watering_logs_plant_date**. On the tiny seed data Postgres may still choose a Seq Scan, because reading a few dozen rows directly is cheaper than using an index. That is correct optimizer behaviour, and a good point to explain if it happens.

11.3 Transactions and ACID

A **transaction** groups SQL statements into one unit: `BEGIN` → statements → `COMMIT` (save all) or `ROLLBACK` (undo all).

Letter	Meaning	Where you can see it in this project
Atomicity	All or nothing	`withTransaction()` in `lib/db/pool.js` does BEGIN / COMMIT, and ROLLBACK on any error. `seed.sql` is one big `BEGIN … COMMIT`. A trigger's work belongs to the same transaction as the INSERT that fired it
Consistency	Data always obeys the rules	Every CHECK, FK, UNIQUE, NOT NULL constraint; a violating statement is rejected and its transaction aborted
Isolation	Concurrent users don't see each other's half-done work	Postgres default level READ COMMITTED, using MVCC (each transaction sees a snapshot)
Durability	Once committed, it survives a crash	Postgres writes the Write-Ahead Log (WAL) to disk before confirming COMMIT

The helper, from `lib/db/pool.js`:

```
export async function withTransaction(transactionFn) {
  const client = await getPool().connect();   // ONE connection for every
step
  try {
    await client.query('BEGIN');
    const result = await transactionFn(client);
    await client.query('COMMIT');
    return result;
  } catch (err) {
    await client.query('ROLLBACK');           // any error → undo everything
    throw err;
  } finally {
    client.release();                         // give the connection back to
the pool
  }
}
```

Why one client? A pool hands out different connections. A transaction lives on *one* connection, so BEGIN, the statements and COMMIT must all go through the same `client`.

Honest note: `withTransaction()` is written but not yet called by any API route (see section 18). The Transactions page in DBMS Studio is an animation, not a live transaction.

12. Security: login, passwords, sessions, roles, SQL injection

Passwords are stored only as bcrypt hashes, login state lives in a signed cookie (JWT), every data route checks who you are, and every query uses `$1`-style parameters. Here is each piece in plain words.

12.1 Registering (`app/api/auth/register/route.js`)

1. The form sends `full_name`, `email`, `password`.
2. The server checks: name not empty, email looks valid, password at least 6 characters.
3. `bcrypt.hash(password, 10)` turns "User@123" into something like `$2b$10$lSV1CE…`. This is **one-way**: you cannot get the password back from the hash. The `10` is the cost factor (2^{10} rounds), which makes guessing slow. Each hash includes a random **salt**, so two users with the same password get different hashes.
4. `INSERT INTO users (email, password_hash, full_name, role_id …) VALUES ($1, $2, $3, 2 …)`. New users are always role 2 (normal user).
5. If Postgres answers error **23505** (duplicate email, from the UNIQUE constraint), the user sees "An account with this email already exists".
6. The user is logged in straight away (a session cookie is set).

12.2 Logging in (`app/api/auth/login/route.js`)

1. `SELECT … FROM users WHERE email = $1 AND is_active = TRUE`.
2. `bcrypt.compare(typedPassword, storedHash)`: hashes the typed password with the stored salt and compares.
3. On success: `UPDATE users SET last_login_at = NOW()` and `INSERT INTO audit_logs (… 'LOGIN' …)`.
4. Creates a **JWT** and puts it in a cookie named `verdebase_session`.
5. Wrong email and wrong password both return the same message, "Invalid email or password", so an attacker cannot learn which emails exist.

12.3 What a JWT is (`lib/auth/session.js`)

A JWT (JSON Web Token) is a small text ticket with three parts: header, data, signature. The data here is `{user_id, email, full_name, role_id}`. The server signs it with `JWT_SECRET` (algorithm HS256) using the `jose` library. Nobody can change the data, e.g. turn role_id 2 into 1, without breaking the signature. It expires after **7 days**.

The cookie is set with:

- `httpOnly: true`: JavaScript in the page cannot read it, which protects against XSS theft.

- `sameSite: 'lax'`: not sent on most cross-site requests, which helps against CSRF.
- `secure` in production: only sent over HTTPS.

No `sessions` table is needed: the server just verifies the signature on each request. That is called **stateless** authentication.

12.4 Checking access on every request

Every data route starts with:

```
const session = await getSession(request);
if (!session) return NextResponse.json({ error: 'Unauthorized' }, { status:
401 });
```

Roles (authorization): `role_id = 1` is admin, `2` is user. The `/api/db/*` routes return **403 Forbidden** unless `role_id === 1`. In data routes the SQL changes with the role:

```
if (session.role_id !== 1) {
  params.push(session.user_id);
  conditions.push(`p.user_id = $${params.length}`);   // normal users only
see their own plants
}
```

So row-level security is done in the application's WHERE clause, not by Postgres itself.

12.5 SQL injection, and why it can't happen here

SQL injection = a user types SQL into an input box and the server pastes it into a query. If the code did `"SELECT * FROM users WHERE email = '" + email + "'"` and someone typed `' OR '1'='1`, they would get every user.

This project never glues user input into SQL. It always sends the SQL and the values **separately**:

```
query('SELECT * FROM users WHERE email = $1', [email]);
```

Postgres treats `$1` purely as a value, never as SQL code. These are called **parameterized queries**. Even the dynamic filters on My Plants push values into the `params` array and only add `$n` placeholders to the text.

12.6 Other protections

- Secrets (DB password, JWT secret, Gmail password, Groq key) live in `.env`, which is listed in `.gitignore`, not in the code.
- DB credentials are only used on the server, never sent to the browser.

- **SQL Lab sandbox:** `/api/db/query` is admin-only and rejects anything not starting with `SELECT` or `EXPLAIN`. There is a gap here, covered in section 18.
- Delete is a **soft delete** (`is_active = FALSE`), so mistakes can be recovered and history is kept.

13. Backend API: every route

The backend is 31 small files in `app/api/`, each handling one URL. Every file follows the same pattern: check the session → read the input → run parameterized SQL through `query()` → return JSON. The HTTP method says what kind of action it is:

- **GET** = read data (SELECT)
- **POST** = create (INSERT)
- **PUT / PATCH** = change (UPDATE)
- **DELETE** = remove (here usually a soft-delete UPDATE)

Status codes you will see: **200** OK, **201** created, **400** bad input, **401** not logged in, **403** logged in but not allowed (not admin), **404** not found, **409** conflict (duplicate), **500** server error.

Authentication

URL	Method	What it does	Main SQL
/api/auth/register	POST	Create an account and log in	`INSERT INTO users …`
/api/auth/login	POST	Check password, set cookie	`SELECT … FROM users WHERE email = $1`; `UPDATE users SET last_login_at`; `INSERT INTO audit_logs`
/api/auth/logout	POST	Delete the cookie	none
/api/auth/me	GET	"Who am I?" for the frontend	reads the user by session

Plant app (logged-in users; admin sees all users' data)

URL	Methods	What it does	Database objects touched
/api/dashboard	GET	Stat cards, recent activity, species pie, 30-day health line	`user_dashboard_stats` view, `plant_activity_summary` view, GROUP BY queries
/api/plants	GET, POST	List (search, filter, pagination with LIMIT/OFFSET) / add a plant	`plants` LEFT JOIN `species`; INSERT fires `trg_after_plant_insert`
/api/plants/[id]	GET, PUT, DELETE	One plant + statistics / edit / soft delete	`get_plant_statistics()` function; `UPDATE plants SET is_active = FALSE`
/api/species	GET, POST	List / add species (POST is admin only)	`species` + `care_profiles` (two INSERTs)
/api/species/[id]	GET, PUT	One species / edit it (admin)	`species`, `care_profiles`
/api/watering	GET, POST	Watering history / log a watering	INSERT fires `trg_after_watering_insert`
/api/fertilization	GET, POST	Fertilizing history / log one	`fertilization_logs`
/api/growth	GET, POST	Growth history / log one	INSERT fires `trg_after_growth_log_insert`
/api/health	GET, POST	Health checks / log one	INSERT fires `trg_after_health_log_insert`
/api/diseases	GET, POST	Disease catalog / add (admin)	`diseases`
/api/disease-records	GET, POST, PATCH	A plant's disease cases / record one / mark resolved	INSERT fires `trg_after_disease_record_insert`

URL	Methods	What it does	Database objects touched
/api/notifications	GET, PATCH, DELETE	List / mark read / remove	`notifications`
/api/reminders (alias /api/reminder)	GET, POST, DELETE	Care schedules	`care_schedules`
/api/analytics?days=30	GET	Summary + trends for N days (1–365)	`get_user_care_summary()` function, `DATE_TRUNC` + GROUP BY
/api/send_reminder	POST	Email any reminders due this minute	`$1 = ANY(schedule_days)` array search
/api/chat	POST	Ask the AI assistant	none (calls Groq)

DBMS Studio (admin only, all return 403 for normal users)

URL	Method	What it returns	Where the data really comes from
/api/db/tables	GET	Every table with row estimate, dead rows, last vacuum/analyze, column and index count	`information_schema.tables`, `pg_stat_user_tables`, `pg_indexes`
/api/db/tables/[name]	GET	Columns and rows of one table	`information_schema.columns` + the table
/api/db/schema	GET	Columns and constraints of all tables (for the ER diagram)	`information_schema.columns`, `table_constraints`
/api/db/views	GET	Every view and its SQL	`information_schema.views`

URL	Method	What it returns	Where the data really comes from
/api/db/functions	GET	Every function with its arguments, return type, full code	`pg_proc`, `pg_get_functiondef()`
/api/db/triggers	GET	Every trigger with event and timing	`information_schema.triggers`
/api/db/indexes	GET	Every index, its size and how often it was used	`pg_indexes`, `pg_stat_user_indexes`
/api/db/statistics	GET	Database size, counts, per-table stats	`get_database_statistics()`, catalogs
/api/db/query	POST	Runs a SELECT/EXPLAIN typed in SQL Lab; returns rows, columns, time	your query
/api/db/explain	POST	Query plan as JSON (optionally ANALYZE inside BEGIN/ROLLBACK)	`EXPLAIN (ANALYZE, FORMAT JSON)`

The system catalog in one line: Postgres keeps a description of itself in special tables (`information_schema.*`, `pg_catalog.pg_*`). Querying them is how DBMS Studio shows the *real* schema instead of a hard-coded picture. This is the database's "data about data", called **metadata**.

14. DBMS Studio: what each page shows

DBMS Studio is 16 admin-only pages that open up the database. 11 of them read live data from PostgreSQL; 4 are teaching pages with fixed content or animation; and the Triggers page reads the live trigger list and animates the event flow. Know which is which before the viva, so you never claim a simulation is live.

Page	URL	What you show the examiner	Live or teaching?
Demo Mode (Viva)	/dbms/demo	A guided menu of cards linking to each concept in a sensible order	Menu
Database Overview	/dbms/overview	DB size, number of tables, views, functions, triggers, indexes	**Live** (`/api/db/statistics`)
ER Explorer	/dbms/er	Draggable ER diagram; click a table to see its columns, PK and FK lines	**Live** (`/api/db/schema` , from `information_schema`)
Tables	/dbms/tables	Every table with row count, columns, indexes, last vacuum; click one to browse its rows	**Live** (`pg_stat_user_tables`)
Relational Model	/dbms/relational-model	Tables written as relation schemas, e.g. PLANTS(<u>plant_id</u>, user_id, species_id, …)	**Live** (`/api/db/schema`)
Normalization	/dbms/normalization	Tabs UNF → 1NF → 2NF → 3NF with the anomalies and functional dependencies	Teaching (fixed content)
SQL Lab	/dbms/sql-lab	Type or pick ready-made queries (basic, aggregation, views, functions, subqueries) and run them	**Live** (`/api/db/query`)
Constraints	/dbms/constraints	A violating INSERT and the Postgres error (code 23514 etc.)	**Simulated** (fixed error text + animation)
Views	/dbms/views	List of the 6 views with their SQL; click to run one and see the result	**Live**
Functions	/dbms/functions	Every function's full PL/pgSQL code; run it	**Live** (`pg_proc`)
Triggers	/dbms/triggers	List of triggers, plus a "Simulate Event Flow" animation INSERT → trigger → UPDATE → notification	List **live**, flow animated
Transactions	/dbms/transactions	BEGIN → steps → COMMIT vs a failing step → ROLLBACK, plus ACID cards	**Simulated** (animation)

Page	URL	What you show the examiner	Live or teaching?
Indexes	/dbms/indexes	Every index with size and number of scans	**Live** (`pg_stat_user_indexes`)
Query Performance	/dbms/query-performance	Plan tree of a SELECT with cost and real time	**Live** (`EXPLAIN ANALYZE`)
Statistics	/dbms/statistics	Row counts and sizes per table	**Live** (`get_database_statistics()`)
Audit Logs	/dbms/audit	Rows of `audit_logs` with their JSONB data	**Live** (via `/api/db/query`)

How to prove a simulated concept live instead

Open SQL Lab (or pgAdmin) and run real statements, so the examiner sees Postgres itself respond.

Constraint (CHECK):

```
INSERT INTO health_logs (plant_id, health_status, health_score)
VALUES (1, 'healthy', 150);
-- ERROR: new row for relation "health_logs" violates check constraint
"health_logs_health_score_check"  (23514)
```

Trigger:

```
SELECT health_status FROM plants WHERE plant_id = 7;        -- before
INSERT INTO disease_records (plant_id, disease_id, severity) VALUES (7, 1,
'severe');
SELECT health_status FROM plants WHERE plant_id = 7;        -- now 'critical'
SELECT * FROM notifications ORDER BY created_at DESC LIMIT 1;  -- new alert
```

Transaction:

```
BEGIN;
INSERT INTO watering_logs (plant_id, quantity_ml) VALUES (1, 250);
INSERT INTO watering_logs (plant_id, quantity_ml) VALUES (1, -5);  -- fails
CHECK
ROLLBACK;   -- the first insert is gone too
```

SQL Lab only accepts SELECT/EXPLAIN, so run INSERTs and BEGIN/ROLLBACK in **pgAdmin's Query Tool** or **psql**.

15. Reminders, notifications and the AI chatbot

Three helper features sit around the core: email reminders run from a schedule table, in-app notifications are written by triggers, and the chatbot passes questions to an AI model. Only the first two touch the database.

15.1 Email reminders

Saving a reminder: on the Reminders page you choose a plant, a task (watering, fertilizing, repotting, pruning, inspection, custom), a time (e.g. 08:00), the days (e.g. Monday, Thursday) and an email. This becomes one row in `care_schedules`, with the days stored as a Postgres **array**: `{monday,thursday}`.

Sending it: a separate background program, `sendReminders.js`, is started with `node sendReminders.js`. Using `node-cron` with the pattern `* * * * *` (every minute), it:

1. Works out the current day ("monday") and time ("08:00").
2. Runs:

```
SELECT cs.*, p.plant_name, u.full_name
FROM care_schedules cs
JOIN users u        ON cs.user_id  = u.user_id
LEFT JOIN plants p  ON cs.plant_id = p.plant_id
WHERE cs.is_active = TRUE
  AND $1 = ANY(cs.schedule_days)                    -- is today in the
array?
  AND TO_CHAR(cs.schedule_time, 'HH24:MI') = $2;    -- is it this minute?
```

3. Emails each matching row through Gmail (nodemailer), e.g. "🌱 VerdeBase - Time to watering My Monstera!".
4. Runs `UPDATE care_schedules SET last_sent_at = NOW()`.

The route `POST /api/send_reminder` does the same job once, so a hosting service's scheduler (e.g. Vercel cron) can call it instead of a long-running process.

What this shows: `ANY()` on an array column, `TO_CHAR` date formatting, a JOIN plus a LEFT JOIN (a reminder may have no plant), and an UPDATE after the send.

15.2 In-app notifications

The `notifications` table is filled **only by triggers**: a new plant, a disease detected, a growth milestone, a health score below 30. The app shell (`AppShell.jsx`) reads `/api/notifications` to show them, and the user can mark them read (PATCH sets `is_read = TRUE`) or delete them. The partial index on unread notifications makes the unread count fast.

The point for the viva: the application code never writes a notification. The database decides, by itself, when the user should be told something.

15.3 The AI assistant

The Chatbot page sends your message to `POST /api/chat`. That route calls **Groq's API** with the model `openai/gpt-oss-20b` and a fixed instruction ("system prompt"): be a friendly household plant-care assistant, answer in concise bullet points with headings and icons, no tables, and advise on watering timing and fertilizer based on pH, soil type, plant type and conditions. The answer comes back and is displayed.

The chatbot is **not** connected to your database: it does not know your plants or logs. It is an add-on feature, not part of the DBMS work. If asked, a good future improvement is to send the plant's latest logs (from `get_plant_statistics()`) along with the question so its advice is personal.

16. Start-to-end walkthroughs of real actions

These five stories trace one click all the way from the button to the database and back. If you can tell these stories, you can explain the whole project.

Story 1: Jane logs in

1. Jane types `jane@verdebase.com` / `User@123` on `/auth` and clicks Login.
2. The browser sends `POST /api/auth/login` with JSON `{"email": "…", "password": "…"}`.
3. The route runs `SELECT user_id, email, password_hash, full_name, role_id FROM users WHERE email = $1 AND is_active = TRUE`.
4. `bcrypt.compare()` matches the password against the stored hash.
5. Two more SQL statements: `UPDATE users SET last_login_at = NOW()` and `INSERT INTO audit_logs (…, 'LOGIN', 'users', …)`.
6. A JWT `{user_id: 2, role_id: 2, …}` is signed and set as the `verdebase_session` cookie.
7. The browser goes to `/dashboard`. From now on every request carries the cookie automatically.

Story 2: Jane opens the dashboard

1. The page calls `GET /api/dashboard`.
2. `getSession()` reads the cookie and verifies the JWT: user 2, not admin.
3. Four SQL queries run:

   a. `SELECT * FROM user_dashboard_stats WHERE user_id = $1` → total plants 8, healthy 5, … (a **view** using `COUNT … FILTER`).
   b. `SELECT * FROM plant_activity_summary WHERE user_id = $1 ORDER BY activity_at DESC LIMIT 5` → last 5 events (a **view** using `UNION ALL`).
   c. A `GROUP BY s.common_name` count → the species pie chart.
   d. `AVG(health_score)` grouped by `DATE(recorded_at)` for 30 days → the health line.
4. The route returns all of it as one JSON object, and Recharts draws the charts.

Story 3: Jane waters her Monstera (the most important story)

1. On the Care page Jane picks "My Monstera", enters 300 ml, and clicks Log.
2. Browser: `POST /api/watering` with `{"plant_id": 1, "quantity_ml": 300, "method": "hand_watering"}`.
3. The route checks the session (401 if missing).
4. **Ownership check:** `SELECT plant_id FROM plants WHERE plant_id = $1 AND user_id = $2 AND is_active = TRUE`. Not hers? 404.
5. **The insert:** `INSERT INTO watering_logs (plant_id, quantity_ml, method, notes, watered_at) VALUES ($1, $2, $3, $4, COALESCE($5::timestamptz, NOW())) RETURNING *`.
6. **Inside PostgreSQL, before the row is saved:** the FK check (does plant 1 exist?) and the CHECKs (`quantity_ml > 0`? method in the allowed list?). A failure here means an error and nothing is saved.
7. **Row saved. `trg_after_watering_insert` fires automatically:**
   a. It calls `calculate_plant_health_score(1)`: watered today, so no watering penalty, then the fertilizer, manual-score and disease rules are applied → e.g. 88.
   b. 88 ≥ 75, so it runs `UPDATE plants SET health_status = 'healthy'`.
   c. That UPDATE fires the BEFORE UPDATE trigger `trg_plants_updated_at`, which sets `updated_at = NOW()`.
   d. It runs `INSERT INTO audit_logs` with `{"plant_id": 1, "computed_health": 88, …}` as JSONB.
8. All of step 5–7 is **one atomic statement**. If the audit insert failed, the watering would also be undone.
9. The route returns `201 Created` with the new log row. The page refreshes, and the Monstera now shows "Healthy" with the next watering date 7 days away.

The JavaScript wrote **one** row. The database did the health calculation, the status change, the timestamp and the audit record.

Story 4: a disease is found on Office ZZ

1. On the plant page, Diseases tab, Jane records "Root Rot", severity "severe".
2. `POST /api/disease-records` → `INSERT INTO disease_records (plant_id, disease_id, detected_at, severity, …)`.
3. Constraints: `disease_id` must exist in `diseases` (FK); severity must be in the list (CHECK).
4. `trg_after_disease_record_insert` fires:
   a. Looks up the plant name and disease name.
   b. Inserts a notification: "⚠️ Disease Detected: Root Rot — Office ZZ has been diagnosed with Root Rot. Severity: severe."
   c. Severity is severe, so `UPDATE plants SET health_status = 'critical'`.
   d. Writes an audit row.
5. Later Jane marks it cured: `PATCH /api/disease-records` → `UPDATE disease_records SET resolved_at = $1 …`. The CHECK `resolved_at >= detected_at` blocks a cure date before the detection date.
6. From then on `disease_summary` shows the case as "resolved" with `days_to_resolve`, and the health function stops deducting 15 points for it.

Story 5 (short): deleting a plant

`DELETE /api/plants/7` does **not** run SQL DELETE. It runs `UPDATE plants SET is_active = FALSE`. The plant disappears from every list and view (they all filter `is_active = TRUE`), but its history stays. If a real `DELETE FROM plants` were run (e.g. in pgAdmin), `ON DELETE CASCADE` would remove all its logs, disease records and schedules, and its notifications would keep existing with `plant_id` set to NULL.

17. Setting up and running the project

You need Node.js 18+ and PostgreSQL 15+; then it is: fill `.env`, run `npm run db:setup`, run `npm run dev`, and open `http://localhost:3000`.

1. **Install** Node.js and PostgreSQL (pgAdmin 4 comes with the Windows installer). Remember the `postgres` user's password.
2. **Open a terminal** in `D:\college\DBMS\plantcare_system\plantcare` and run `npm install` (downloads the libraries into `node_modules/`).
3. **Create `.env`** by copying `.env.example`, then fill in:
   - `DB_HOST=localhost`, `DB_PORT=5432`, `DB_NAME=verdebase`, `DB_USER=postgres`, `DB_PASSWORD=<your postgres password>`

- `JWT_SECRET=<any long random text, 32+ characters>`
- optional: `EMAIL_USER` / `EMAIL_PASS` (a Gmail **app password**) for reminders, `NEXT_PUBLIC_GROQ_API_KEY` for the chatbot

4. **Build the database:** `npm run db:setup`. This runs `database/init.js`, which:
   a. Connects to the default `postgres` database and runs `CREATE DATABASE verdebase` if it doesn't exist.
   b. Runs the SQL files in dependency order: `schema.sql` → `indexes.sql` → `views.sql` → `functions.sql` → `triggers.sql` → `seed.sql`.

   The order matters: views need tables, the watering trigger needs the health function, and the seed needs triggers so it generates notifications and audit rows.
   Manual alternative: in psql or pgAdmin, `CREATE DATABASE verdebase;` then run the six files in the same order.
5. **Start the website:** `npm run dev`, then open `http://localhost:3000`.
6. **Log in:** admin `admin@verdebase.com` / `Admin@123` (sees DBMS Studio) or user `jane@verdebase.com` / `User@123`.
7. **Optional, for reminders:** in a second terminal, `node sendReminders.js`.

Common problems

Symptom	Likely cause	Fix
`password authentication failed for user "postgres"`	Wrong `DB_PASSWORD` in `.env`	Fix the password and restart `npm run dev`
`ECONNREFUSED 127.0.0.1:5432`	PostgreSQL service is not running	Start it from Windows Services or pgAdmin
`relation "plants" does not exist`	Database not set up	`npm run db:setup`
`function calculate_plant_health_score does not exist` when watering	`functions.sql` wasn't run, or ran after `triggers.sql`	Re-run setup in order
Login fails with the seed accounts	Using `password123` (the README is wrong)	Use `Admin@123` / `User@123`
DBMS Studio pages say Forbidden	Logged in as Jane	Log in as admin
Chatbot says "Sorry, something went wrong"	No Groq key	Add the key to `.env`

Useful scripts (`package.json`)

Command	What it does
`npm run dev`	Development server with live reload (Turbopack)
`npm run build` then `npm start`	Production build and run
`npm run db:setup`	Create and fill the database
`npm run lint`	Check code style

18. Honest limitations, and how to fix them

The project is solid, but reading the code turned up 11 issues; an examiner could notice about half of them. Each one comes with what to say and the fix. Fixing the first five before the viva is worth it; most take minutes.

#	Issue	Where	Why it matters	Fix
1	README says the primary keys are UUIDs; the schema uses `SERIAL` integers	`README.md` table section	Contradiction if the examiner reads both	Change "UUID" to "SERIAL" in the README
2	README says the seed password is `password123`; it is really `Admin@123` / `User@123` (checked against the hashes)	`README.md` §4	Login fails during the demo	Correct the README
3	Adding a species runs two separate INSERTs (`species` , then `care_profiles`) without a transaction; `withTransaction()` exists but is never used	`app/api/species/route.js` , `lib/db/pool.js`	If the second INSERT fails, a species is left without a care profile: a broken "all or nothing"	Wrap both INSERTs in `withTransaction(async (client) => { … })` . This also gives you a **real** transaction to show
4	The Transactions and Constraints pages are animations, while the README says "Zero Fake Features"	`app/dbms/transactions` , `app/dbms/constraints`	Overclaiming is the easiest thing to get caught on	Call them "visual explainers" and show the real thing in pgAdmin (section 14)
5	Dashboard cards "Critical" and "Tasks Today" always show 0	`app/dashboard/page.jsx` reads `stats.critical` and `stats.care_tasks_today` ; the view returns `critical_plants` and has no tasks column	Visible wrong number on the first screen	Read `critical_plants` ; add a tasks-today count to the view or remove the card
6	SQL Lab only checks that the text *starts* with SELECT. With no parameters, node-postgres allows several statements in one string, so `SELECT 1; DROP TABLE x;` would pass	`app/api/db/query/route.js`	Admin-only, but still a hole in the "read-only sandbox" claim	Reject any `;` before the end, or run inside `BEGIN READ ONLY … ROLLBACK` , or use a DB user with SELECT-only rights

#	Issue	Where	Why it matters	Fix
7	`idx_users_email` duplicates the index Postgres already built for `UNIQUE(email)`	`indexes.sql`	Classic viva question: "why index a UNIQUE column?"	Remove it, or explain it as redundant
8	In the health-log trigger, `>= 20 → critical` and `ELSE → critical` are the same, so the branch does nothing	`triggers.sql` trigger 4	Dead code	Merge the branches, or map < 20 to something else
9	The Groq key is named `NEXT_PUBLIC_…`; in Next.js that prefix can expose a variable to the browser	`.env`, `app/api/chat/route.js`	Leaks the API key if it is ever used client-side	Rename to `GROQ_API_KEY`
10	`/api/chat` and `/api/send_reminder` do not check login	those routes	Anyone can use your AI quota or trigger emails	Add `getSession()`, or a secret header for the cron route
11	Row access is enforced in app code (WHERE user_id = …), not in Postgres; the watering route's ownership check uses the session's own user ID, so even an admin can't log watering for another user's plant	API routes	Fine for this scope	Future: PostgreSQL Row-Level Security policies

Design limits worth admitting (they are choices, not bugs)

- `plants.health_status` is stored, not derived, so it is updated only when a log is inserted. A plant that is never watered again does not "get worse" by itself until the next event. A nightly job or computing it in a view would fix that.
- `symptoms TEXT[]` and `schedule_days TEXT[]` are arrays, a small departure from strict 1NF chosen for simplicity.
- The `user_dashboard_stats` view joins plants × waterings × notifications and relies on `COUNT(DISTINCT)`, which gets slow with large data. Separate subqueries would scale better.
- `@supabase/supabase-js` and `lib/supabaseClient.js` are leftovers from version 1 and can be removed.

Future scope (good to say at the end of a viva)

PostgreSQL Row-Level Security; a materialized view for analytics refreshed nightly; table partitioning of the log tables by month; connecting the chatbot to the user's own plant data; a mobile app; IoT soil-moisture sensors writing directly into `watering_logs`.

19. Viva questions with model answers

These are the questions an examiner is most likely to ask about this project, with short answers in your own project's terms.

About the project

1. **What is your project?** A plant-care management system on a normalized 14-table PostgreSQL database. Users log watering, fertilizing, growth, health and diseases; the database computes plant health with functions and triggers; and an admin DBMS Studio lets us inspect the schema, run queries and see execution plans live.
2. **Why is this a DBMS project and not just a website?** Because the rules live in the database: constraints validate data, views answer the dashboard, PL/pgSQL functions compute health, triggers update status and write notifications and audit logs. The website mostly inserts one row and reads views.
3. **Why PostgreSQL and not MySQL or MongoDB?** We need strict relations, FKs, CHECKs, triggers and PL/pgSQL, plus Postgres extras: LATERAL joins, `FILTER` on aggregates, arrays, JSONB, and good EXPLAIN output. MongoDB has no joins or FK enforcement in this sense.
4. **Which DBMS concepts does it cover?** ER modelling, the relational model, keys, all constraint types, referential actions, normalization to 3NF, joins of every kind, aggregates and GROUP BY, subqueries, UNION, views, stored functions, triggers, indexes, query plans, transactions/ACID, system catalogs, auditing.

ER, keys and constraints

5. **What are the entities and relationships?** Users, roles, species, care profiles, plants, four log types, diseases, disease records, schedules, notifications, audit logs. User–plant 1:N, species–plant 1:N, species–care_profile 1:1, plant–logs 1:N, plant–disease M:N through `disease_records`.
6. **How did you make species–care_profile one-to-one?** The FK `care_profiles.species_id` also has a UNIQUE constraint, so a species can appear only once.
7. **How is an M:N relationship stored?** With a junction table holding two FKs: `disease_records(plant_id, disease_id)`, plus relationship attributes like severity, detected_at, resolved_at.
8. **Difference between primary key, candidate key and foreign key, with examples?** PK `plant_id`; candidate key `users.email` (unique, could identify a user); FK `plants.user_id` referencing `users`.

9. **Why surrogate (SERIAL) keys instead of email or scientific name?** They are small, never change, and make joins fast; natural keys are still kept unique with UNIQUE constraints.
10. **What happens when you delete a user? A species? A disease?** User: CASCADE deletes their plants and logs. Species: plants stay with species_id SET NULL. Disease in use: RESTRICT blocks the delete.
11. **Give a CHECK constraint and what happens if violated.** `health_score BETWEEN 0 AND 100`; inserting 150 fails with SQLSTATE 23514 and nothing is saved.
12. **What is the composite unique constraint for?** `UNIQUE(user_id, nickname)`: a nickname must be unique per user, not across all users.

Normalization

13. **Up to which normal form is it?** 3NF.
14. **Show a transitive dependency you removed.** `plant_id → species_id → watering_interval_days`. The interval was moved to `care_profiles`, keyed by species.
15. **What anomalies did normalization remove?** Update (change Monstera's interval in one row, not every plant), insert (add a species with no plants), delete (removing a plant doesn't remove its owner or species).
16. **Is anything denormalized?** Yes, deliberately: `plants.health_status` and `growth_stage` are stored for fast reads and kept consistent by triggers. The arrays `symptoms` and `schedule_days` are a small 1NF compromise.
17. **Is it in BCNF?** For the main tables yes: every determinant is a key (e.g. in `species`, both `species_id` and `scientific_name` are keys).

SQL, views, functions, triggers

18. **What is a view, and is its data stored?** A saved SELECT; no data is stored, it runs on current data each time. (A *materialized* view would store it.)
19. **What is a LATERAL join, and where is it used?** A subquery in FROM that can reference the outer row; used in `plant_health_summary` to get each plant's latest health, watering and growth log.
20. **UNION vs UNION ALL?** UNION removes duplicates (extra sorting); UNION ALL keeps all rows and is faster. `plant_activity_summary` uses UNION ALL because events from different tables are never duplicates.
21. **WHERE vs HAVING?** WHERE filters rows before grouping; HAVING filters groups after aggregation.

22. **What does `COUNT(*) FILTER (WHERE …)` do?** Counts only rows matching the condition inside one aggregate. Used in `user_dashboard_stats` to count healthy, critical, etc. in one pass.
23. **Function vs procedure vs trigger?** A function returns a value or table and is called in SQL; a procedure is called with CALL and can control transactions; a trigger function runs automatically on an event and returns NEW/OLD.
24. **Explain your health-score function.** Starts at 100; −15/−30 for overdue watering vs species interval; −10 if not fertilized in 60 days; blends 60/40 with the latest manual score; −15 per active disease; clamps to 0–100.
25. **What is NEW in a trigger?** The row being inserted or updated. (OLD is the previous version, in UPDATE/DELETE.)
26. **BEFORE vs AFTER trigger, and why each here?** BEFORE UPDATE sets `updated_at` before the row is written; AFTER INSERT triggers need the saved row (its ID) and change other tables.
27. **Can triggers cause other triggers?** Yes: the watering trigger updates `plants`, which fires the `updated_at` trigger.
28. **If the trigger fails, is the watering saved?** No. The trigger runs in the same transaction as the INSERT, so everything rolls back.

Indexes and performance

29. **What is an index and which type do you use?** A B-tree structure that finds rows without scanning the table; all of ours are B-tree.
30. **Why index foreign keys?** Postgres doesn't do it automatically, and joins and per-user filters use them constantly.
31. **What is a composite index, and the leftmost-prefix rule?** An index on several columns, e.g. `(plant_id, watered_at DESC)`; it helps queries filtering by plant_id first, not by watered_at alone.
32. **What is a partial index?** An index with a WHERE: `notifications(user_id, is_read) WHERE is_read = FALSE` only stores unread rows.
33. **Why might EXPLAIN show Seq Scan even with an index?** On small tables a sequential read is cheaper; the optimizer picks the cheapest plan using table statistics.
34. **Disadvantages of indexes?** Slower INSERT/UPDATE and extra disk space.
35. **EXPLAIN vs EXPLAIN ANALYZE?** EXPLAIN shows the estimated plan; ANALYZE actually runs it and shows real times. We wrap ANALYZE in BEGIN/ROLLBACK so it can't change data.

Transactions and concurrency

36. **Explain ACID with your project.** Atomicity: a watering + its trigger's updates are all-or-nothing. Consistency: constraints. Isolation: Postgres MVCC, READ COMMITTED. Durability: WAL.
37. **Where do you use transactions?** `withTransaction()` in `pool.js`, `BEGIN…COMMIT` around the seed, `BEGIN…ROLLBACK` around EXPLAIN ANALYZE, and every single statement is its own implicit transaction.
38. **What is MVCC?** Multi-Version Concurrency Control: readers see a snapshot and don't block writers; each update creates a new row version.
39. **What is a connection pool and why use one?** A set of reusable open DB connections (max 10 here) so each request doesn't pay the cost of connecting.

Security

40. **How do you store passwords?** bcrypt hashes with salt and cost factor 10; never plain text.
41. **How do you prevent SQL injection?** Parameterized queries: SQL text with `$1` placeholders and values sent separately.
42. **How is login maintained?** A signed JWT in an HTTP-only cookie, valid 7 days, checked on every API call.
43. **How does a user only see their own plants?** Every query adds `WHERE p.user_id = $n` with the logged-in user's ID unless the user is admin.
44. **What is the audit log for, and why JSONB?** A history of important changes and logins; JSONB stores a flexible snapshot of each changed row, whatever its columns.

Catalogs and the studio

45. **How does DBMS Studio know the schema?** It queries the system catalogs: `information_schema.tables/columns/table_constraints/views/triggers`, `pg_proc`, `pg_indexes`, `pg_stat_user_tables`.
46. **What is `n_dead_tup` / VACUUM?** Dead tuples are old row versions left by MVCC; VACUUM cleans them up and ANALYZE refreshes statistics for the optimizer.

Tricky ones

47. **Is `idx_users_email` needed?** No. UNIQUE already creates an index; it is redundant.
48. **Is the transactions page live?** It is a visual explainer. The real transactions are in `pool.js`, the seed script and the explain route, and I can demonstrate one in pgAdmin.

49. **What would you improve?** Use `withTransaction` for species creation, Row-Level Security in Postgres, a materialized view for analytics, partitioning of log tables, and making the chatbot use the user's own data.
50. **What was hardest?** Getting "latest log per plant" efficiently (LATERAL + composite DESC indexes) and making triggers chain correctly without loops.